Assignment 6

SRE Automation and Capacity Planning for a Containerized Microservices System

Docker Compose | Nginx | Flask Microservices | PostgreSQL 15 | Prometheus | Grafana | cAdvisor | node-exporter | Terraform

Student	Kaber Daryn
Report Date	05 May 2026
Scope	Evidence-backed Assignment 6 report covering SRE automation, monitoring, alerting, configuration validation, capacity planning, scaling strategy, Terraform linkage, and self-healing recovery.

Evidence boundary. This report validates a local Docker Compose SRE deployment. It does not claim production e-commerce feature completeness, cloud Terraform apply execution, or Kubernetes auto-scaling. Terraform configuration is initialized and validated locally; scaling policies are proposed from measured behavior.

1. Executive Summary

The final Assignment 6 implementation extends the microservices system with deployment automation, health checks, self-healing behavior, monitoring, alerting, log inspection, configuration validation, capacity analysis, and scaling recommendations. The active runtime uses PostgreSQL 15 and excludes obsolete MySQL artifacts from the final submission package.

Area	Evidence-backed result
Runtime automation	Docker Compose deploys all application, database, ingress, and monitoring services with health checks and restart policies.
Database layer	PostgreSQL 15.17 is verified; product records are stored in PostgreSQL and returned by Product API.
Monitoring and alerting	Prometheus scrapes application, container, and host metrics; alert rules are loaded and promtool validates 11 rules.
Dashboarding	Grafana has a working Prometheus datasource and a dashboard with target availability, CPU, and memory panels.
Capacity planning	Controlled tests capture requests, RPS, error rate, average latency, p95, p99, and SLO boundary behavior.
Recovery	Order Service controlled crash recovers through Docker restart policy; RestartCount changes from 0 to 1 and /health returns 200.
IaC linkage	Terraform files are present and terraform init/validate succeed through the Terraform Docker image.

The main capacity limit observed in the local environment is latency growth, not request failure. The capacity section therefore defines an SLO-based boundary: p95 latency below 300 ms and error rate below 1%.

2. Requirement Coverage

The mapping below connects Assignment 6 requirements to the implemented evidence. Section references refer to this report.

Requirement	Implementation evidence	Where shown
Dockerized microservices context	Frontend, User/Auth, Product, Order, PostgreSQL, Nginx, Prometheus, Grafana, cAdvisor, node-exporter.	Fig. 2
Automated deployment	Root Docker Compose creates a repeatable multi-container deployment.	Fig. 2, Fig. 10
Health checks and self-healing	Services expose /health; Docker health checks and restart policies recover Order Service after failure.	Fig. 8, Fig. 10, Fig. 26
Monitoring-based alerting	Prometheus scrape config, alert rules, alert UI, promtool, Grafana, and cAdvisor are included.	Fig. 11-18
Log-based troubleshooting automation	Automated log inspection checks application and monitoring containers for critical runtime patterns.	Fig. 9
Configuration validation	Validation script checks required files, endpoints, Docker service status, and Prometheus readiness.	Fig. 8, Fig. 10
Capacity analysis	Load matrix, boundary test, resource snapshots, and CPU stress evidence are included.	Fig. 19-25
Scaling strategy	Scaling actions are tied to measured latency, error rate, CPU, PostgreSQL pressure, and restart counts.	Section 14
Terraform integration	Terraform files exist and terraform init/validate succeed. Cloud apply is not claimed.	Fig. 27-28

3. Architecture

The architecture is organized into four separate planes: ingress, application services, PostgreSQL relational storage, and observability. The primary validated request path is Browser -> Nginx -> Flask Frontend -> Product Service -> PostgreSQL Product DB. User/Auth and Order services are deployed independently and monitored through the same SRE stack.

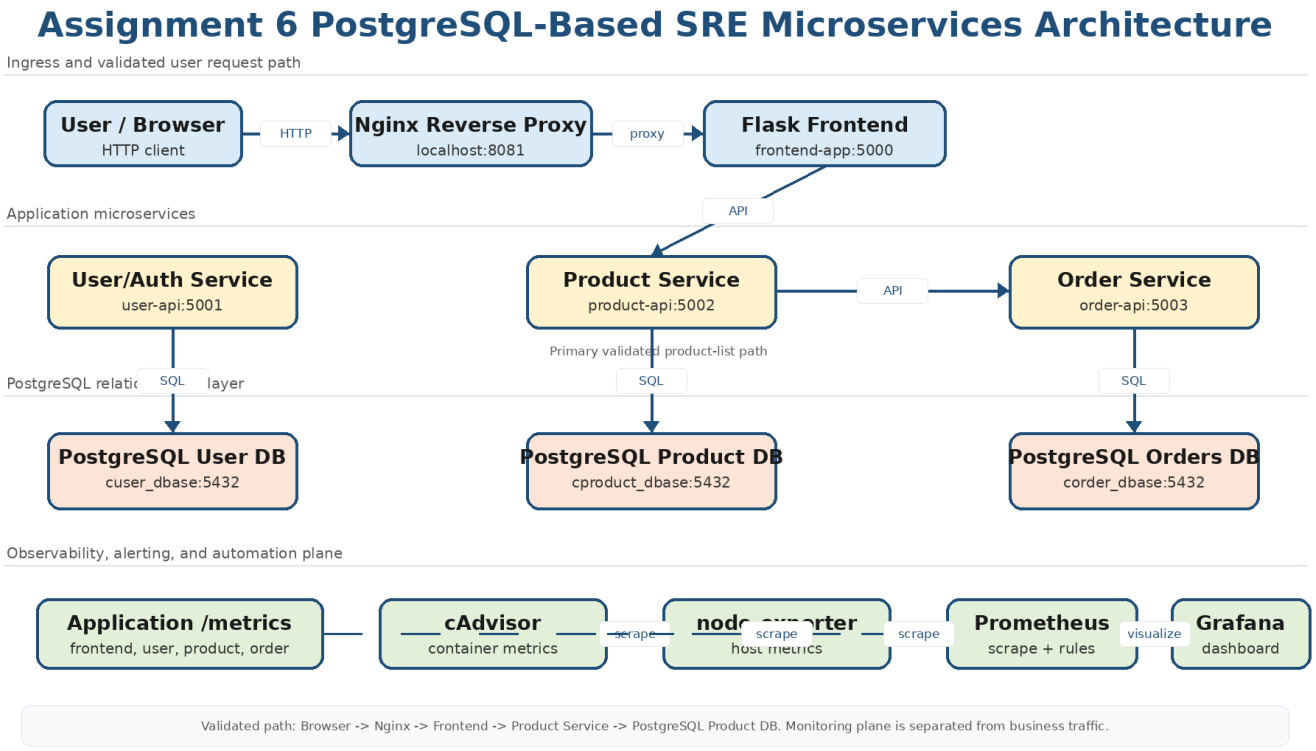

Figure 1. Clean PostgreSQL-based architecture for Assignment 6.

The observability plane is intentionally separate from the user request path. Prometheus scrapes application /metrics endpoints, cAdvisor, and node-exporter. Grafana visualizes the Prometheus datasource. This avoids coupling user traffic to monitoring components.

4. Runtime Deployment and PostgreSQL Evidence

The deployment evidence shows the full runtime stack with PostgreSQL 15 databases, application services, Nginx, Prometheus, Grafana, cAdvisor, and node-exporter.

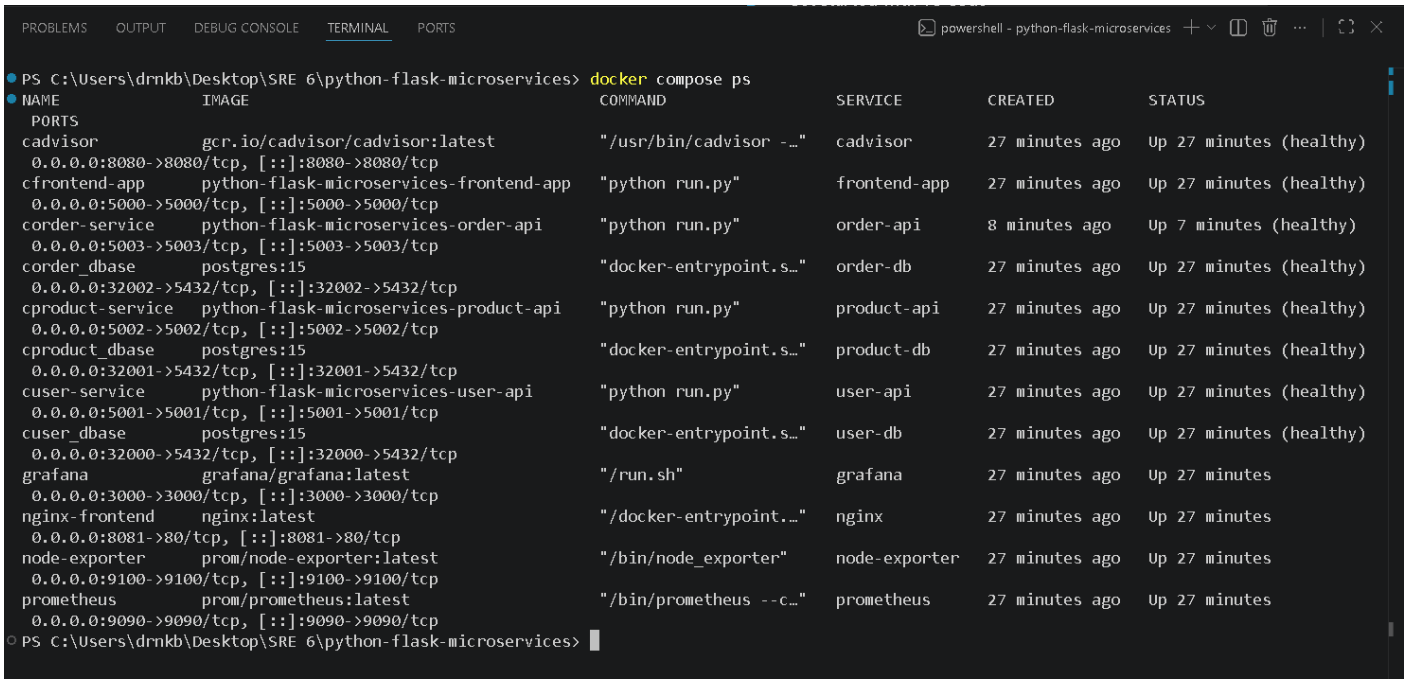

Figure 2. Docker Compose stack: PostgreSQL 15 database containers and application/monitoring services are running.

5. PostgreSQL Data Path Validation

PostgreSQL is not only present as a container; it is used by the Product API path. The database version and product records are verified through psql, then returned by the Product API and rendered through the Nginx frontend.

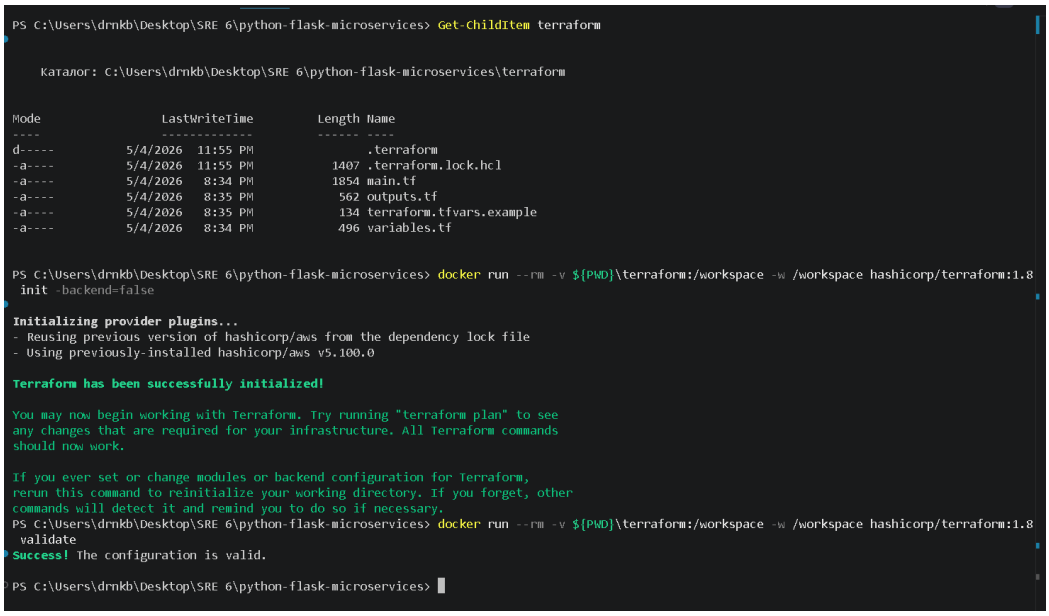

Figure 3. PostgreSQL version evidence: PostgreSQL 15.17 is running in the Product database container.

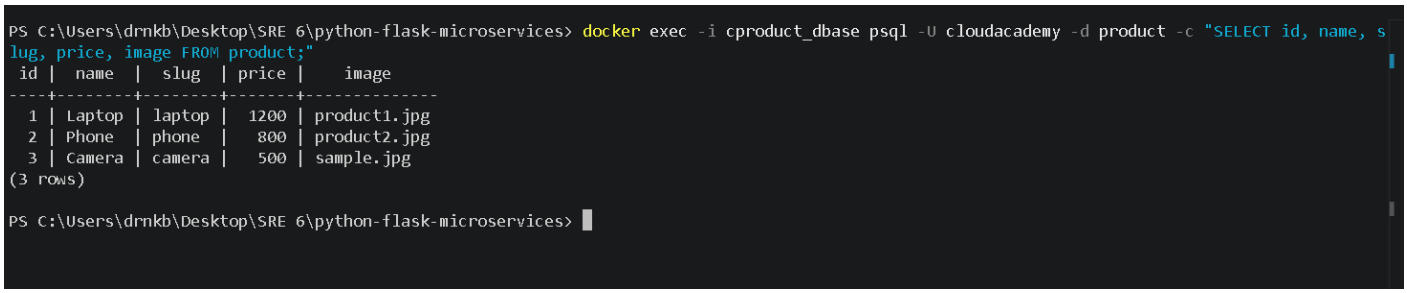

Figure 4. PostgreSQL product table contains the records used by the Product API.

Figure 5. Product API returns HTTP 200 with product JSON backed by PostgreSQL records.

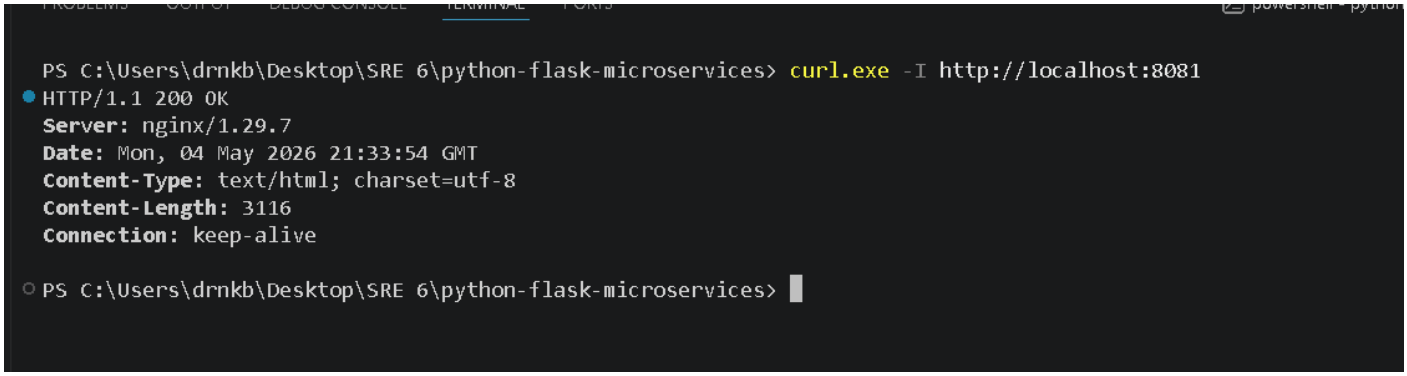

Figure 6. Nginx frontend endpoint returns HTTP 200 through localhost:8081.

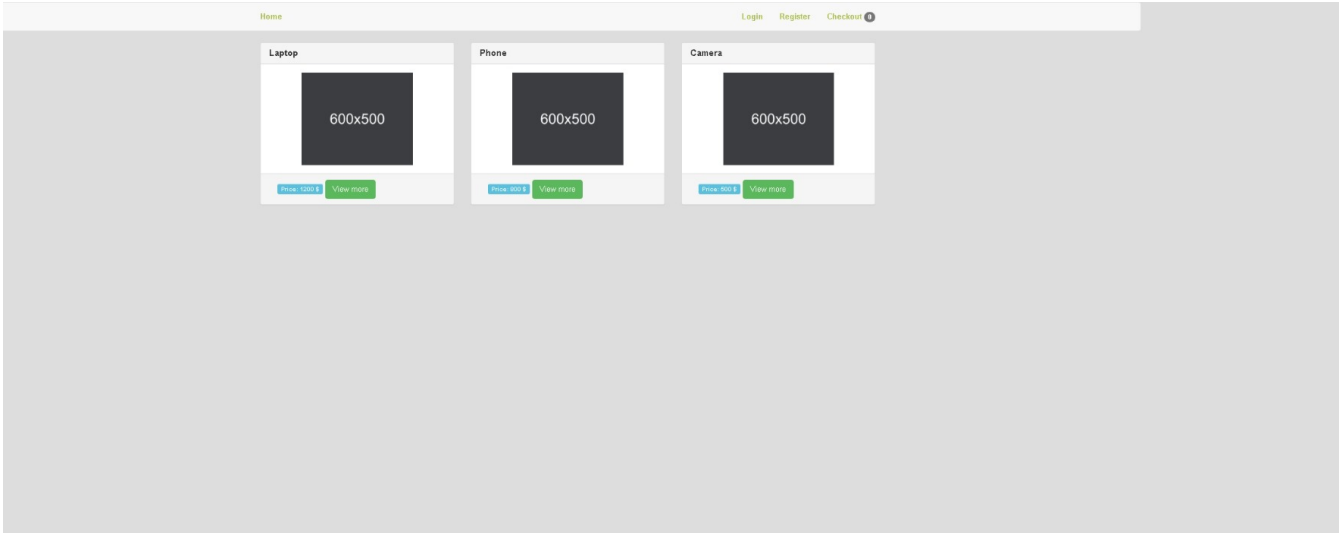

Figure 7. Browser evidence: frontend page renders product cards through the validated path.

6. Validation and Log Automation

The validation script checks required files, running endpoints, Docker service status, and Prometheus readiness. The log inspection script automates troubleshooting by scanning application and monitoring logs for critical error patterns.

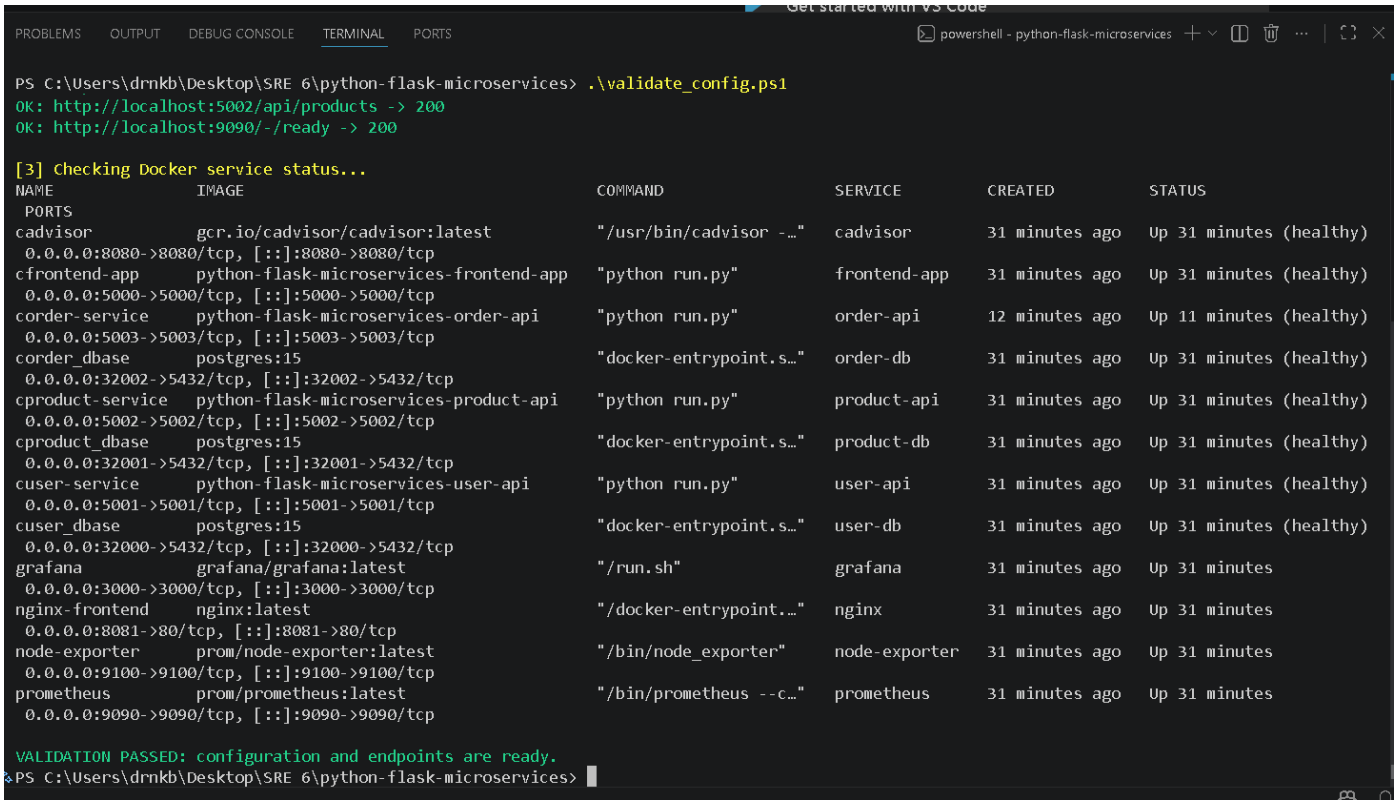

Figure 8. Pre-deployment/runtime validation: required files, service endpoints, Docker status, and Prometheus readiness pass.

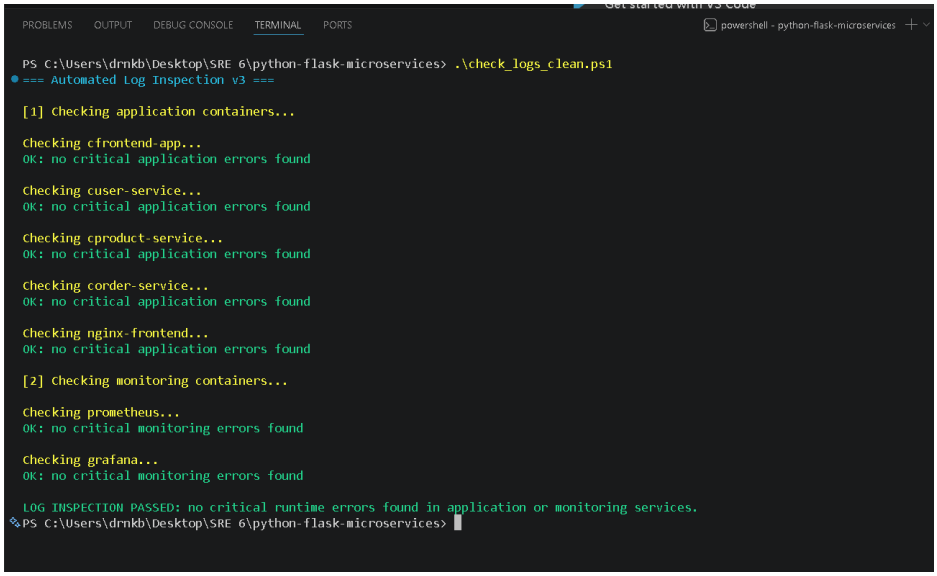

Figure 9. Automated log inspection reports no critical application or monitoring runtime errors.

7. Configuration Evidence

Runtime screenshots prove that the system is running. This section shows configuration-level evidence that the behavior is actually encoded in the project files: PostgreSQL, restart policies, health checks, service dependencies, Prometheus scrape jobs, and alert rules.

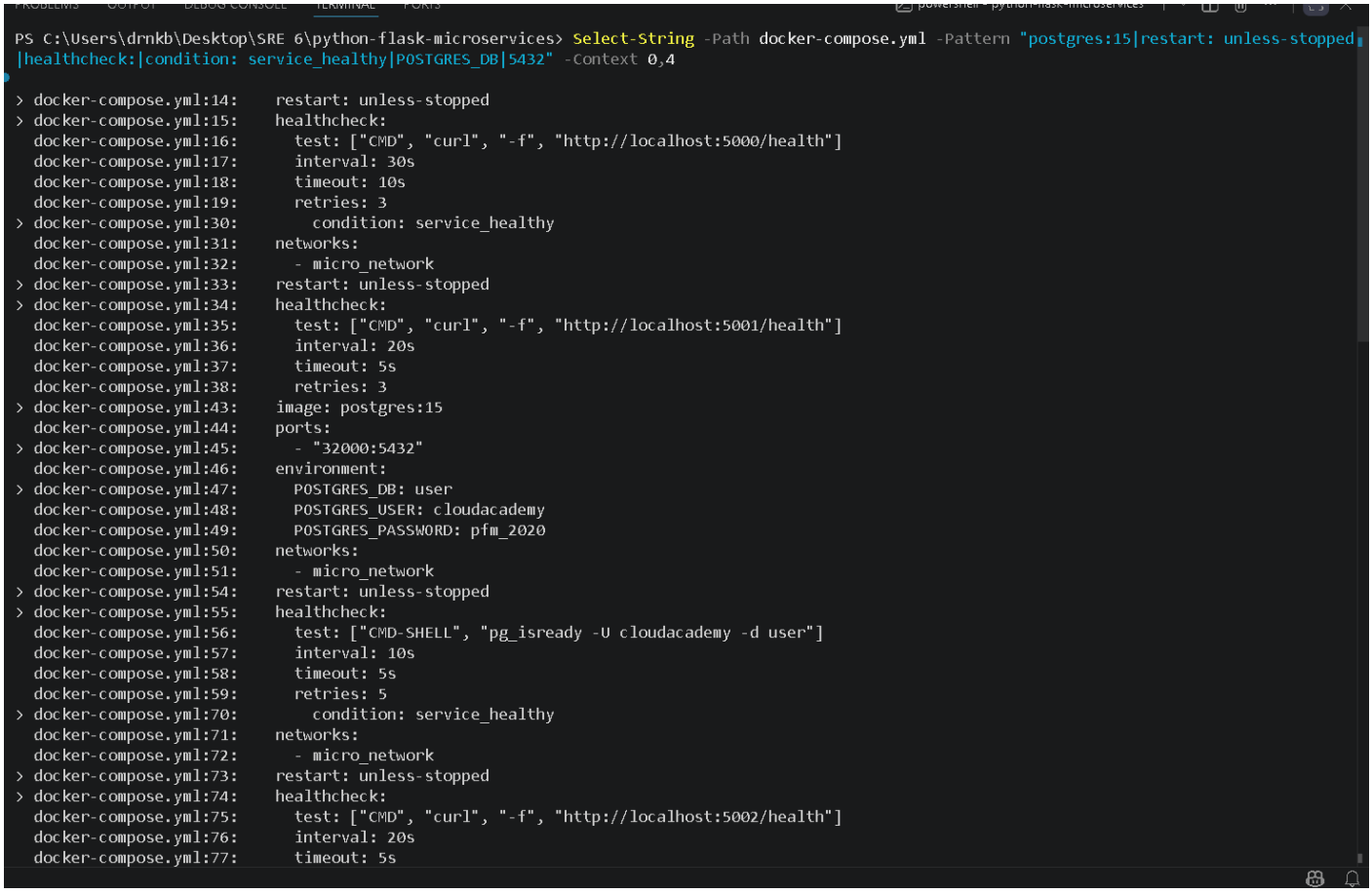

Figure 10. docker-compose.yml evidence: restart policies, health checks, service_healthy dependencies, PostgreSQL 15, and port 5432.

```
PS C:\Users\drnkb\Desktop\SRE 6\python-flask-microservices> Get-Content monitoring\prometheus\prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - "alert_rules.yml"

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']

  - job_name: 'user-service'
    static_configs:
      - targets: ['cuser-service:5001']
    metrics_path: '/metrics'

  - job_name: 'product-service'
    static_configs:
      - targets: ['cproduct-service:5002']
    metrics_path: '/metrics'

  - job_name: 'order-service'
    static_configs:
      - targets: ['corder-service:5003']
    metrics_path: '/metrics'

  - job_name: 'frontend'
    static_configs:
      - targets: ['cfrontend-app:5000']
    metrics_path: '/metrics'
PS C:\Users\drnkb\Desktop\SRE 6\python-flask-microservices>
```

Figure 11. Prometheus scrape configuration: Prometheus, node-exporter, cAdvisor, and application /metrics jobs.

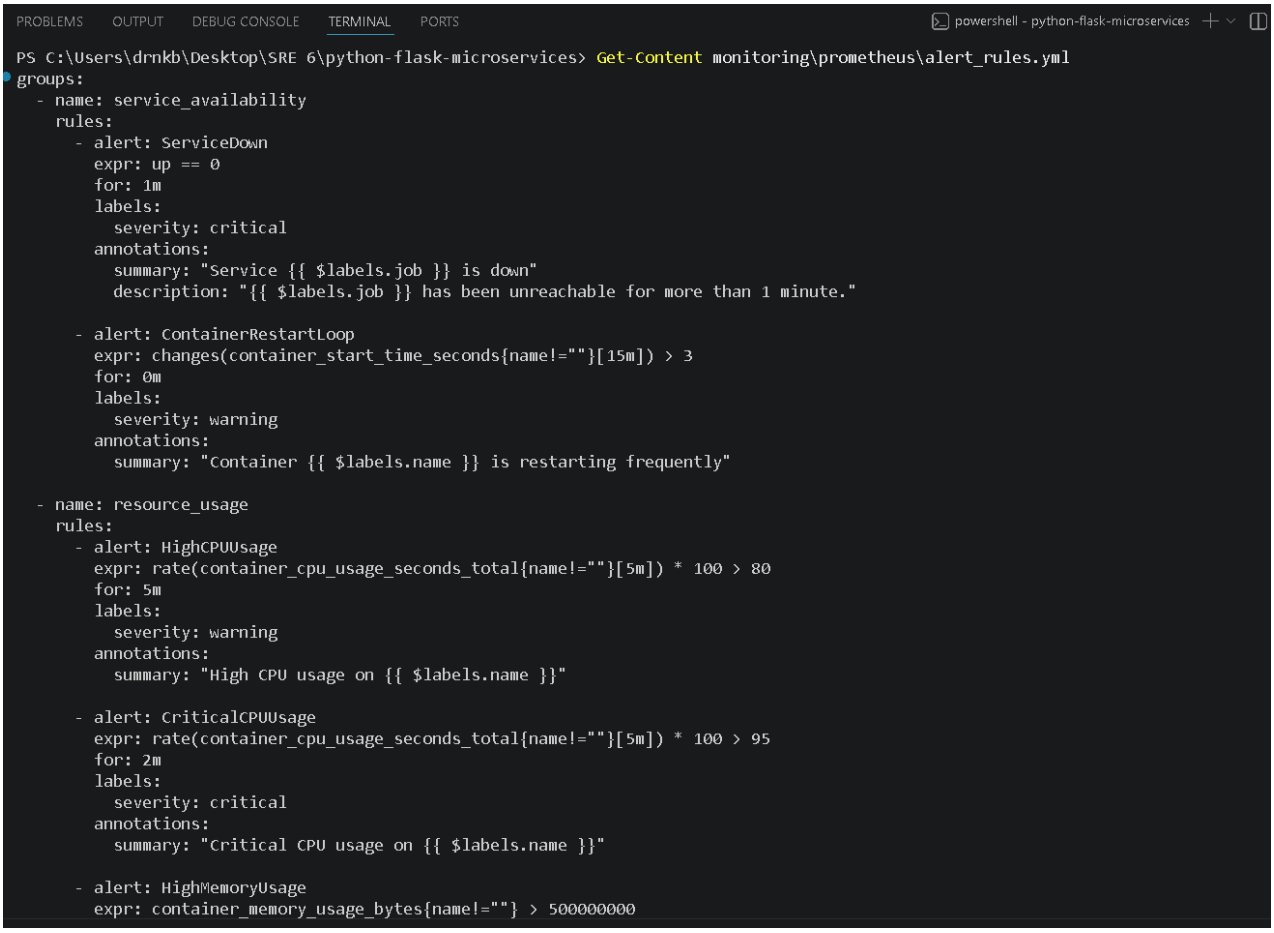

Figure 12. Alert rule configuration: service availability, restart loop, CPU, and memory alert examples.

The following table explains representative alert rules so that alerting evidence is not only a screenshot of loaded rules.

Alert rule	Condition / threshold	SRE purpose
ServiceDown	up == 0 for 1 minute	Detect service outage across Prometheus jobs.
ContainerRestartLoop	changes(container_start_time_seconds[15m]) > 3	Detect repeated container restarts.
HighCPUUsage	container CPU rate * 100 > 80 for 5 minutes	Warn on sustained CPU pressure.
CriticalCPUUsage	container CPU rate * 100 > 95 for 2 minutes	Critical alert for CPU saturation risk.
HighMemoryUsage	container_memory_usage_bytes > 500,000,000	Warn on memory saturation risk.

8. Monitoring and Alerting Evidence

The monitoring stack includes Prometheus targets, Prometheus alert rules, Grafana datasource/dashboard, cAdvisor container metrics, and node-exporter host metrics.

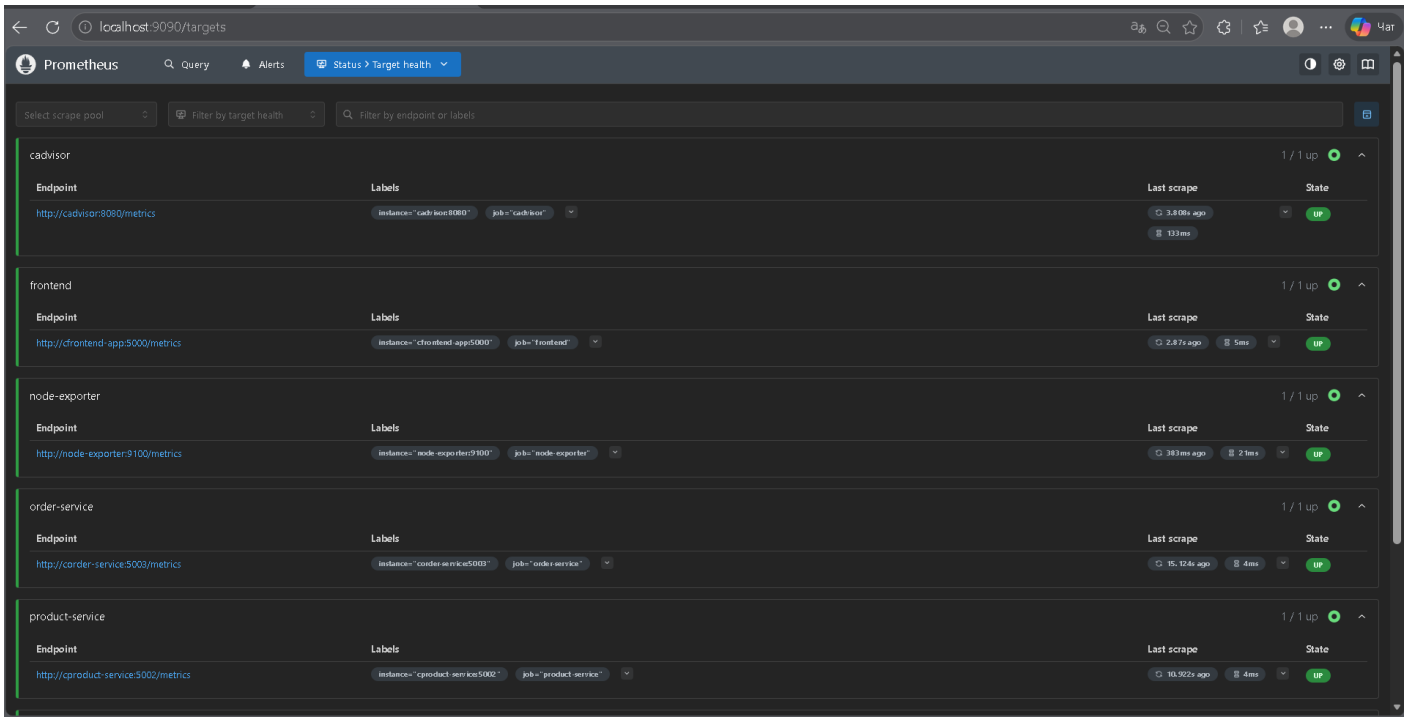

Figure 13. Prometheus targets are UP for service, host, and container metric sources.

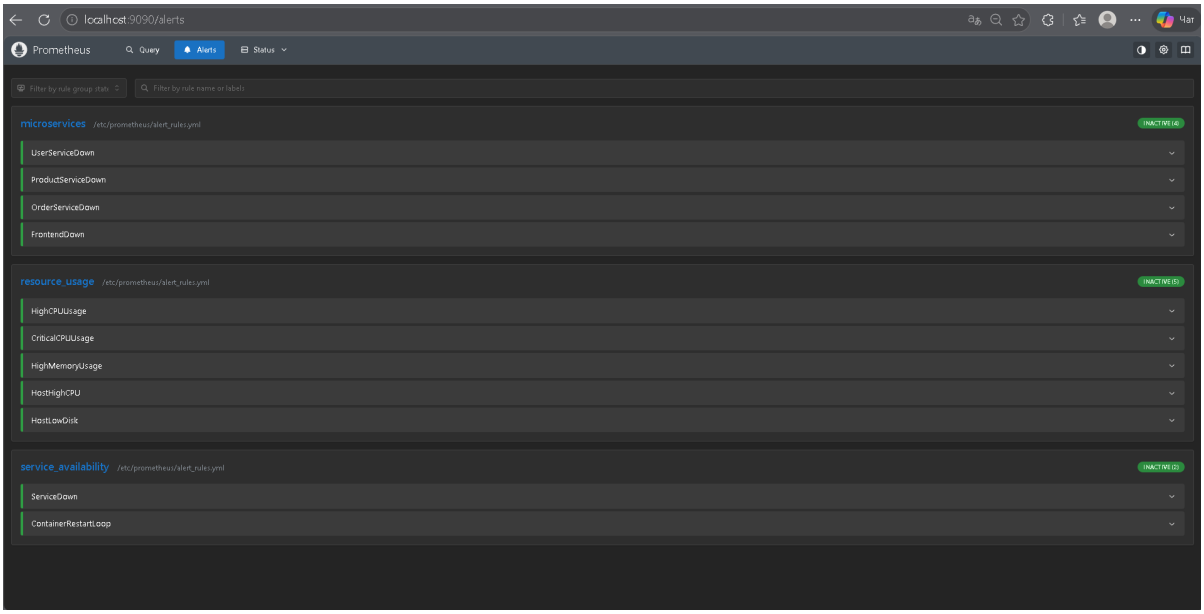

Figure 14. Prometheus alert groups are loaded in the Alerts UI.

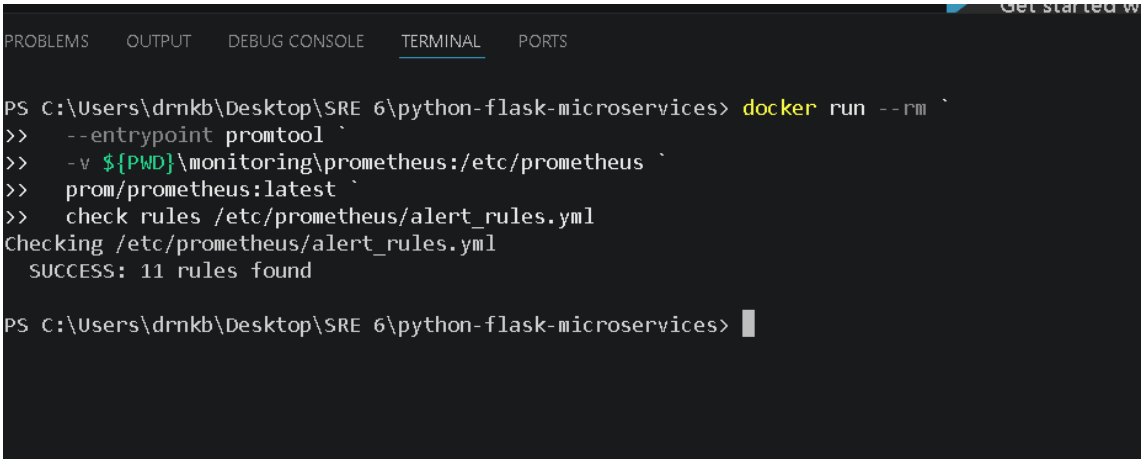

Figure 15. promtool validation succeeds: 11 alert rules found.

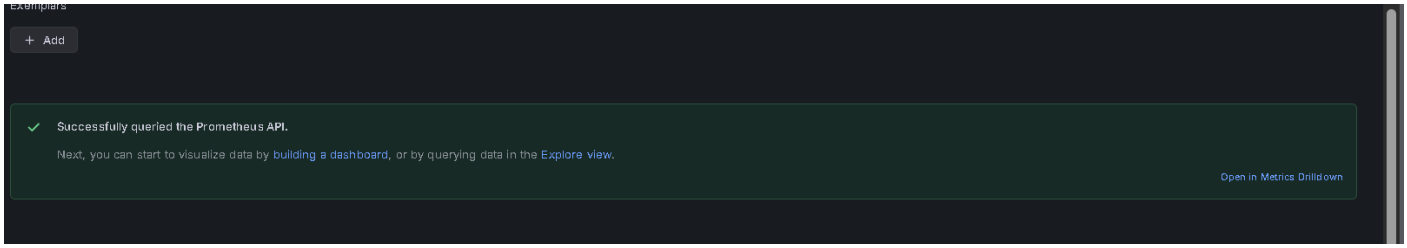

Figure 16. Grafana datasource evidence: Prometheus API is successfully queried.

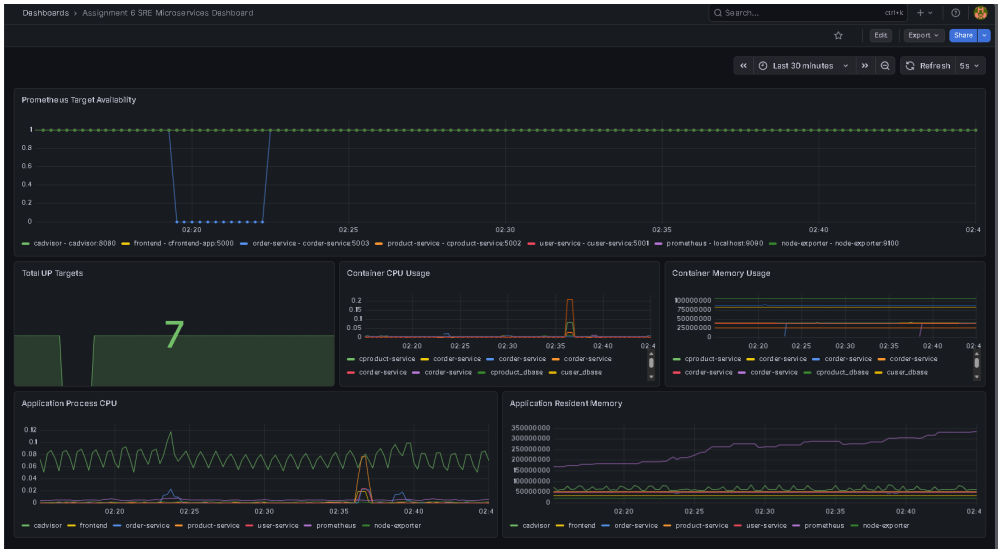

Figure 17. Grafana dashboard visualizes target availability, CPU, and memory panels.

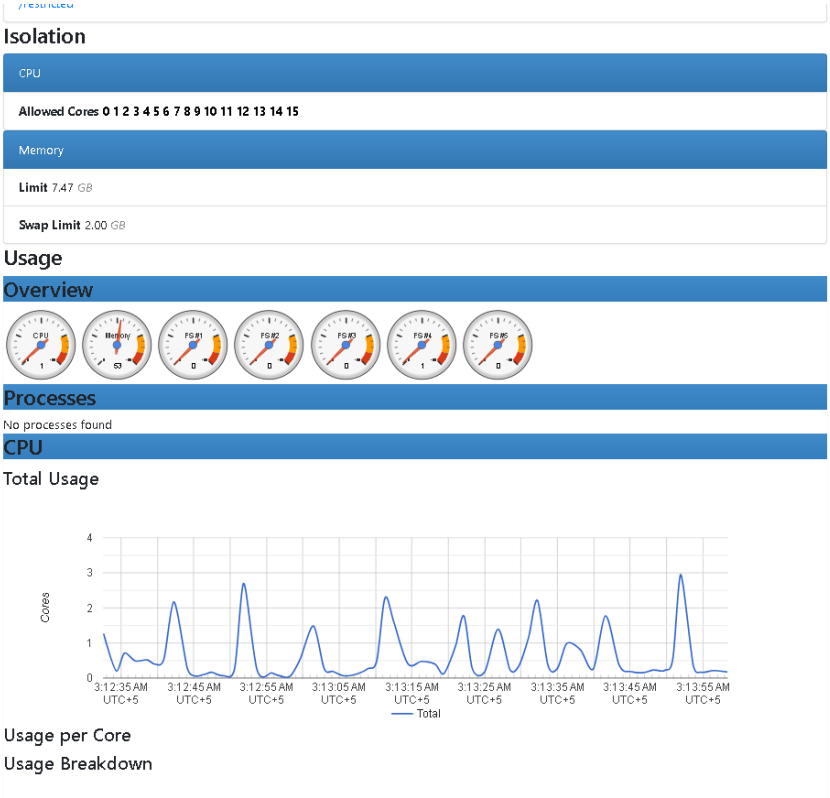

Figure 18. cAdvisor evidence: container-level metrics are available for Prometheus scraping.

9. Capacity Planning Method

The load simulation uses concurrent requests against frontend health, backend health endpoints, and the database-backed Product API. This measures request success, error rate, RPS, average latency, p95, and p99. The report also defines a local SLO for practical capacity analysis: p95 latency below 300 ms and error rate below 1%.

Test	Concurrency	Requests	Success	Failed	Error rate	RPS	Avg	P95	P99
Baseline	5	250	250	0	0.00%	377.38	0.0131s	0.0216s	0.0260s
Medium	20	500	500	0	0.00%	398.03	0.0466s	0.0795s	0.0904s
High	50	750	750	0	0.00%	383.67	0.1077s	0.1909s	0.2092s
Stress	100	1000	1000	0	0.00%	335.27	0.1906s	0.4738s	0.5519s

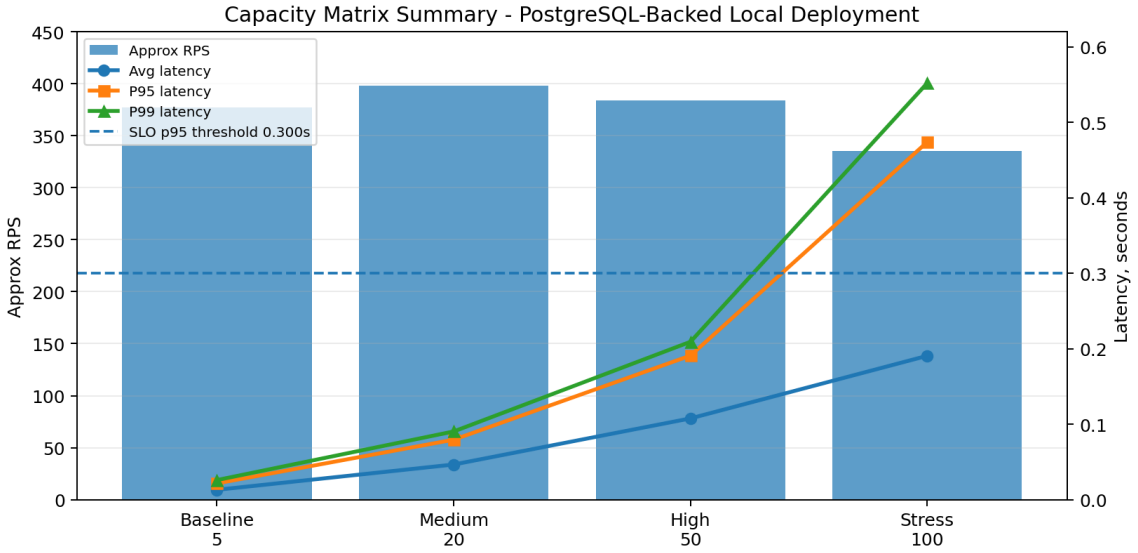

Figure 19. Capacity matrix chart: latency increases sharply under stress even with zero failed requests.

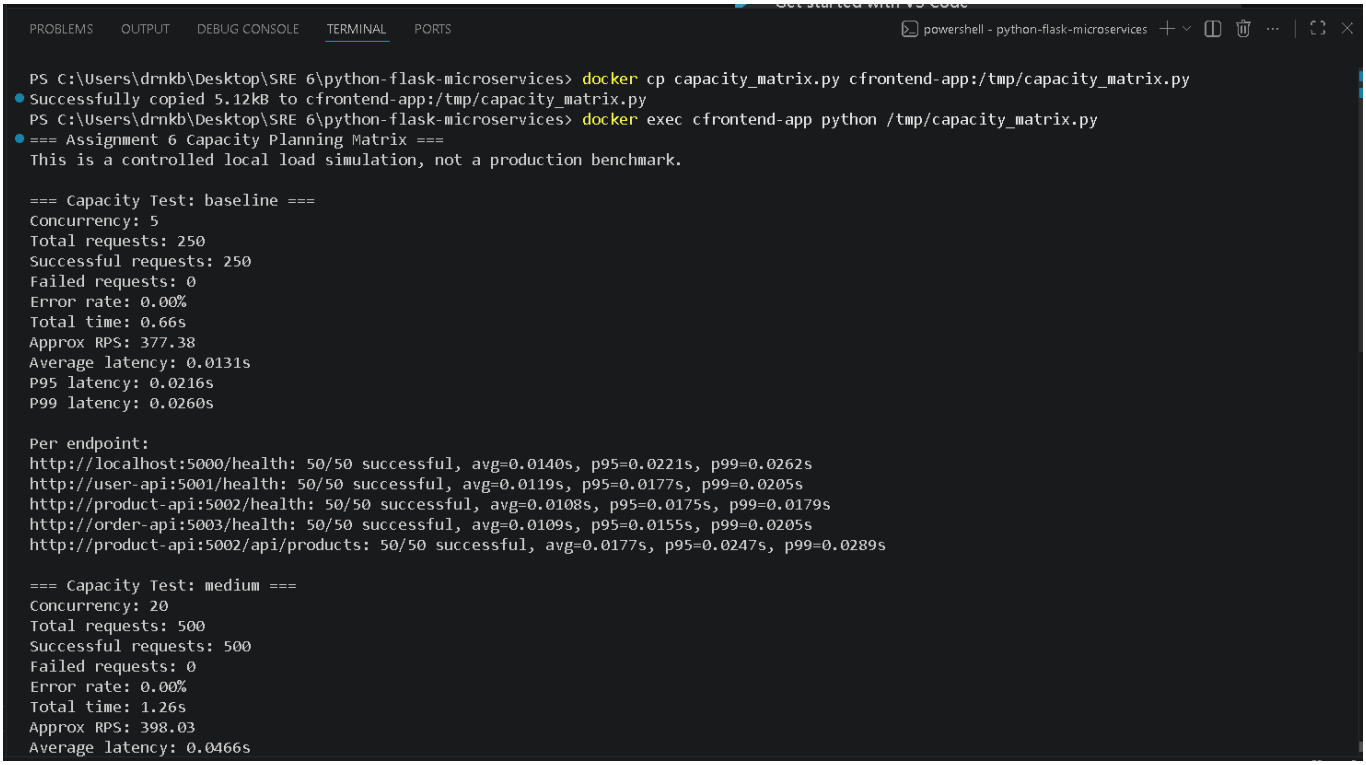

Figure 20. Raw capacity matrix evidence: baseline and medium load test results.

```
PS C:\Users\drnkb\Desktop\SRE 6\python-flask-microservices> docker exec cfrontend-app python /tmp/capacity_matrix.py
=== Capacity Test: high ===
Concurrency: 50
Total requests: 750
Successful requests: 750
Failed requests: 0
Error rate: 0.00%
Total time: 1.95s
Approx RPS: 383.67
Average latency: 0.1077s
P95 latency: 0.1909s
P99 latency: 0.2092s

Per endpoint:
http://localhost:5000/health: 150/150 successful, avg=0.0638s, p95=0.1471s, p99=0.2255s
http://user-api:5001/health: 150/150 successful, avg=0.1023s, p95=0.1545s, p99=0.1857s
http://product-api:5002/health: 150/150 successful, avg=0.1058s, p95=0.1703s, p99=0.2117s
http://order-api:5003/health: 150/150 successful, avg=0.1068s, p95=0.1810s, p99=0.2092s
http://product-api:5002/api/products: 150/150 successful, avg=0.1599s, p95=0.2025s, p99=0.2078s

=== Capacity Test: stress ===
Concurrency: 100
Total requests: 1000
Successful requests: 1000
Failed requests: 0
Error rate: 0.00%
Total time: 2.98s
Approx RPS: 335.27
Average latency: 0.1906s
P95 latency: 0.4738s
P99 latency: 0.5519s

Per endpoint:
http://localhost:5000/health: 200/200 successful, avg=0.0555s, p95=0.1409s, p99=0.1833s
```

Figure 21. Raw capacity matrix evidence: high and stress load test results.

10. Capacity Boundary and SLO Analysis

A system can have zero request failures and still violate a latency objective. For that reason, the practical capacity boundary is defined by p95 latency and error rate, not by outright request failure alone. The local SLO assumption used here is p95 < 0.300 seconds and error rate < 1%.

Boundary run	Concurrency	Requests	Error rate	Approx RPS	Avg latency	P95	P99	SLO result
boundary-60	60	600	0.00%	282.98	0.1633s	0.3926s	0.4954s	FAIL
boundary-70	70	700	0.00%	328.11	0.1549s	0.2996s	0.3901s	PASS
boundary-80	80	800	0.00%	359.46	0.1552s	0.3004s	0.3839s	Edge / breach

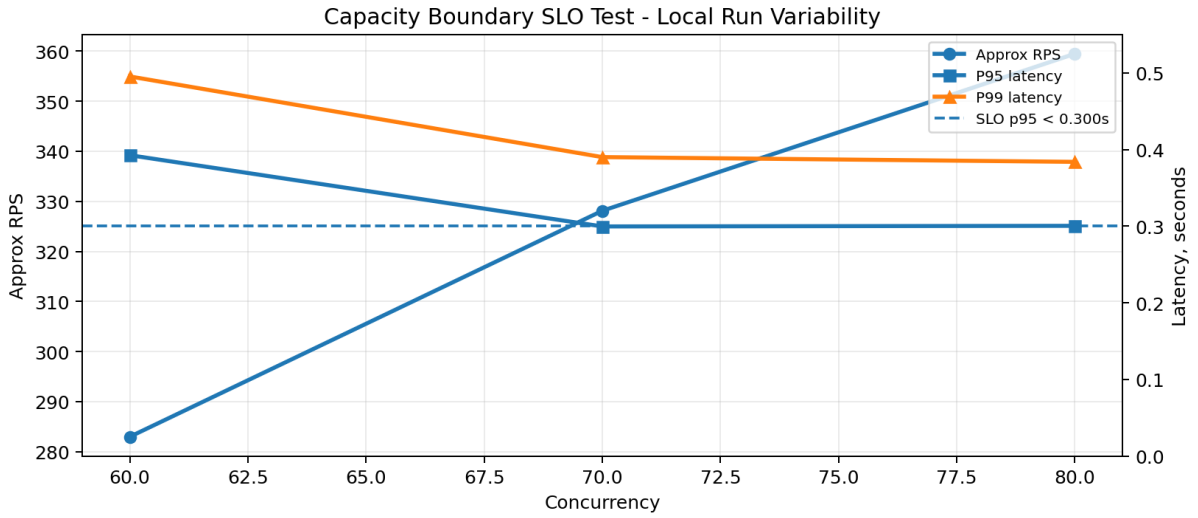

Figure 22. Boundary chart: p95 approaches or crosses the 300 ms SLO near the 60-80 concurrency range.

Interpretation. Local Docker Desktop load results are noisy, so the boundary is not perfectly monotonic. The conservative conclusion is that SLO risk starts before the concurrency-100 stress scenario. Since stress-100 has p95 0.4738s, scaling should begin before concurrency 100 if p95 < 300 ms is required.

```
>>
PS C:\Users\drnkb\Desktop\SRE 6\python-flask-microservices> docker cp capacity_boundary_test.py cfrontend-app:/tmp/capacity_boundary_test.py
Successfully copied 4.1kB to cfrontend-app:/tmp/capacity_boundary_test.py
PS C:\Users\drnkb\Desktop\SRE 6\python-flask-microservices> docker exec cfrontend-app python /tmp/capacity_boundary_test.py
=== Assignment 6 Capacity Boundary SLO Test ===
SLO assumption: p95 < 0.300s and error rate < 1.00%

=== Capacity Boundary Test: boundary-60 ===
Concurrency: 60
Total requests: 600
Successful requests: 600
Failed requests: 0
Error rate: 0.00%
Approx RPS: 282.98
Average latency: 0.1633s
P95 latency: 0.3926s
P99 latency: 0.4954s
SLO result: FAIL

=== Capacity Boundary Test: boundary-70 ===
Concurrency: 70
Total requests: 700
Successful requests: 700
Failed requests: 0
Error rate: 0.00%
Approx RPS: 328.11
Average latency: 0.1549s
P95 latency: 0.2996s
P99 latency: 0.3901s
SLO result: PASS

=== Capacity Boundary Test: boundary-80 ===
Concurrency: 80
Total requests: 800
Successful requests: 800
Failed requests: 0
Error rate: 0.00%
Approx RPS: 359.46
Average latency: 0.1552s
P95 latency: 0.3004s
P99 latency: 0.3839s
```

Figure 23. Raw capacity boundary SLO test: p95 threshold and SLO pass/fail output.

11. Resource Usage and CPU Stress

Capacity analysis is supported by container-level resource snapshots and a controlled CPU stress test. These measurements do not claim production capacity; they provide local evidence for which services are candidates for scaling and tuning.

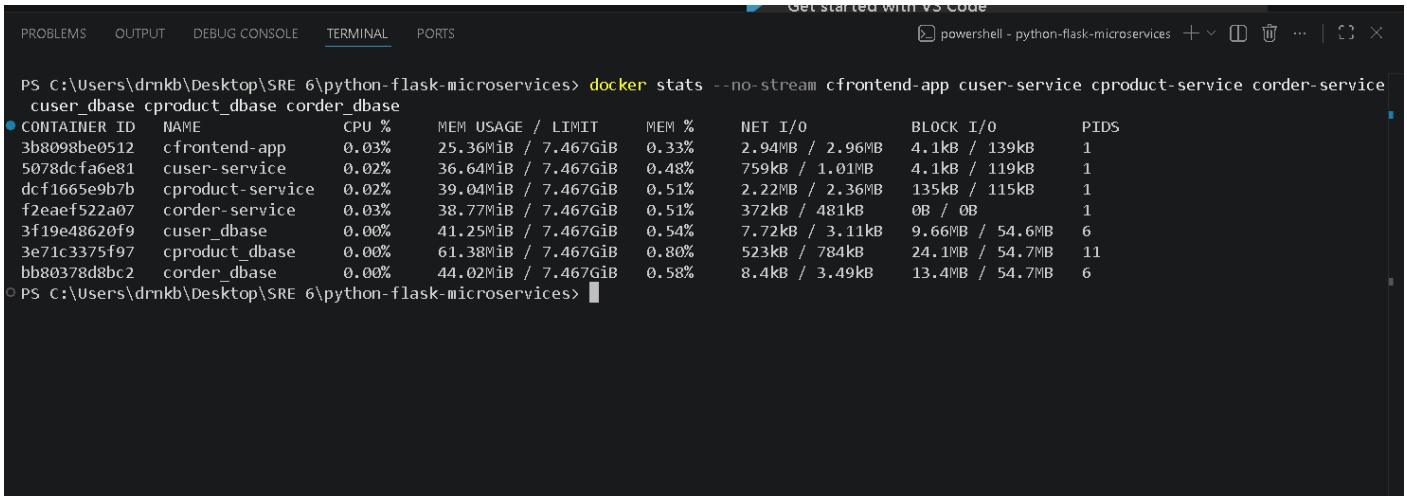

Figure 24. docker stats: CPU and memory usage for application services and PostgreSQL containers.

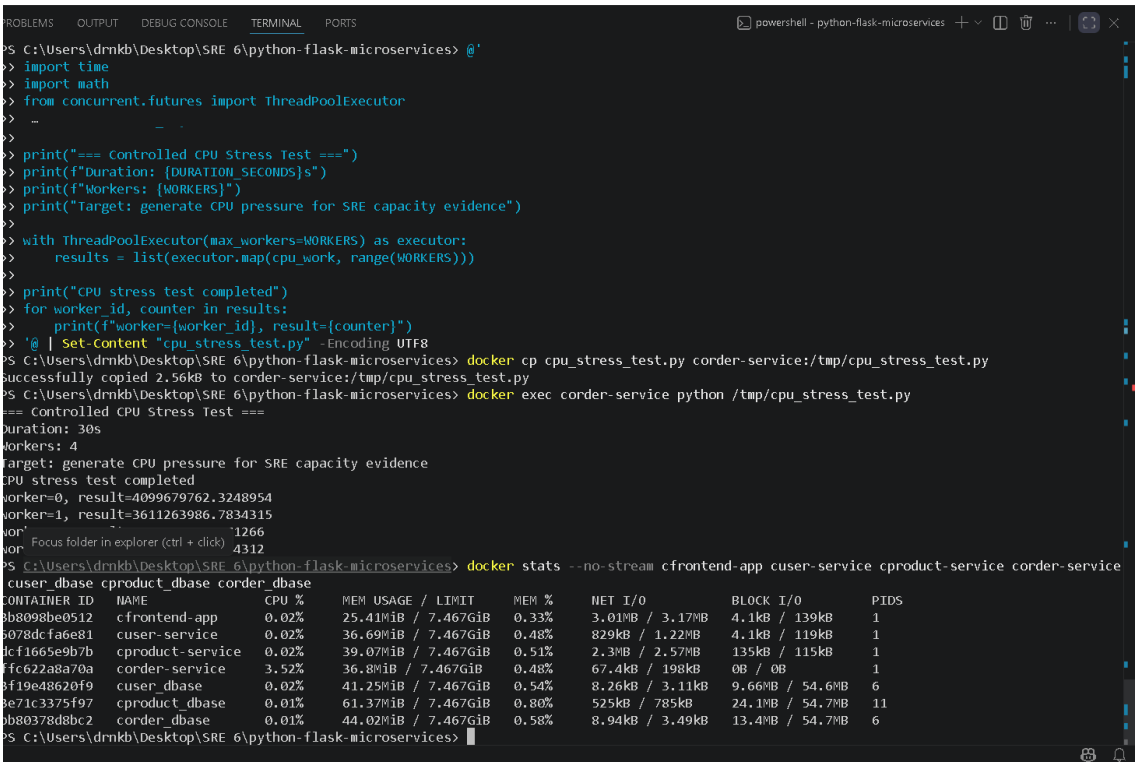

Figure 25. Controlled CPU stress evidence with docker stats snapshot.

Resource interpretation. Product API is the database-backed read path and shows latency sensitivity under repeated load. Order Service remains the resilience target because it was the incident-linked service and is used in recovery validation. PostgreSQL containers are included in monitoring and stats because database saturation is a likely future bottleneck.

12. Self-Healing and Recovery

The recovery demonstration uses a controlled failure-injection endpoint in the Order Service. This endpoint is test instrumentation only and is not presented as a production business endpoint. The evidence shows process-level failure, Docker restart policy recovery, RestartCount increase, and health endpoint recovery.

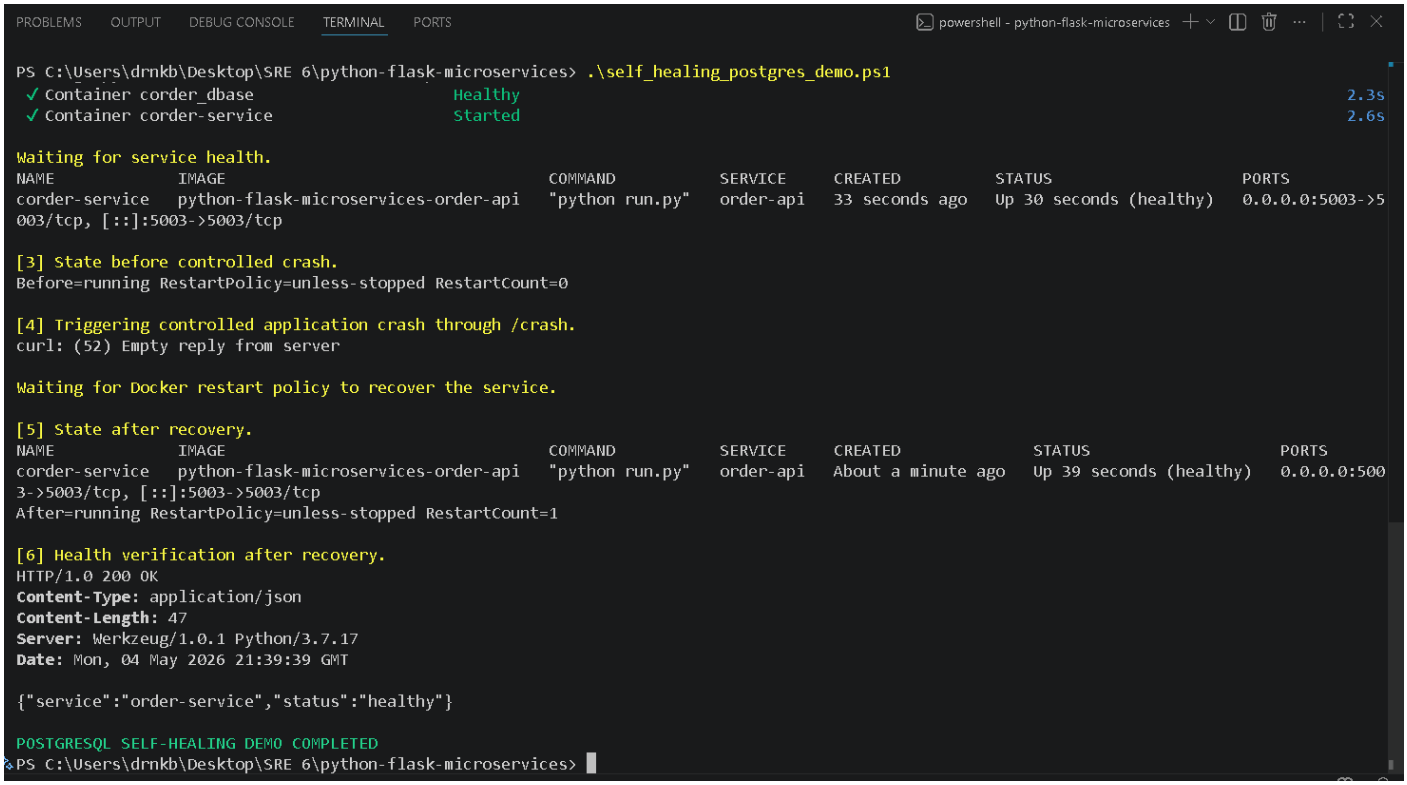

Figure 26. Self-healing evidence: controlled crash, Docker restart policy recovery, RestartCount 0 -> 1, and /health returns HTTP 200.

13. Terraform / Infrastructure-as-Code Linkage

Terraform files are included for Assignment 5 linkage and Assignment 6 infrastructure context. The configuration is initialized and validated through the Terraform Docker image. Cloud provisioning is not claimed because terraform apply was not executed in the local evidence scope.

```
PS C:\Users\drnkb\Desktop\SRE 6\python-flask-microservices> Get-ChildItem terraform


    Каталог: C:\Users\drnkb\Desktop\SRE 6\python-flask-microservices\terraform


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         5/4/2026  11:55 PM                .terraform
-a----         5/4/2026  11:55 PM           1407 .terraform.lock.hcl
-a----         5/4/2026   8:34 PM           1854 main.tf
-a----         5/4/2026   8:35 PM            562 outputs.tf
-a----         5/4/2026   8:35 PM            134 terraform.tfvars.example
-a----         5/4/2026   8:34 PM            496 variables.tf


PS C:\Users\drnkb\Desktop\SRE 6\python-flask-microservices>
```

Figure 27. Terraform directory evidence: main.tf, variables.tf, outputs.tf, and terraform.tfvars.example are present.

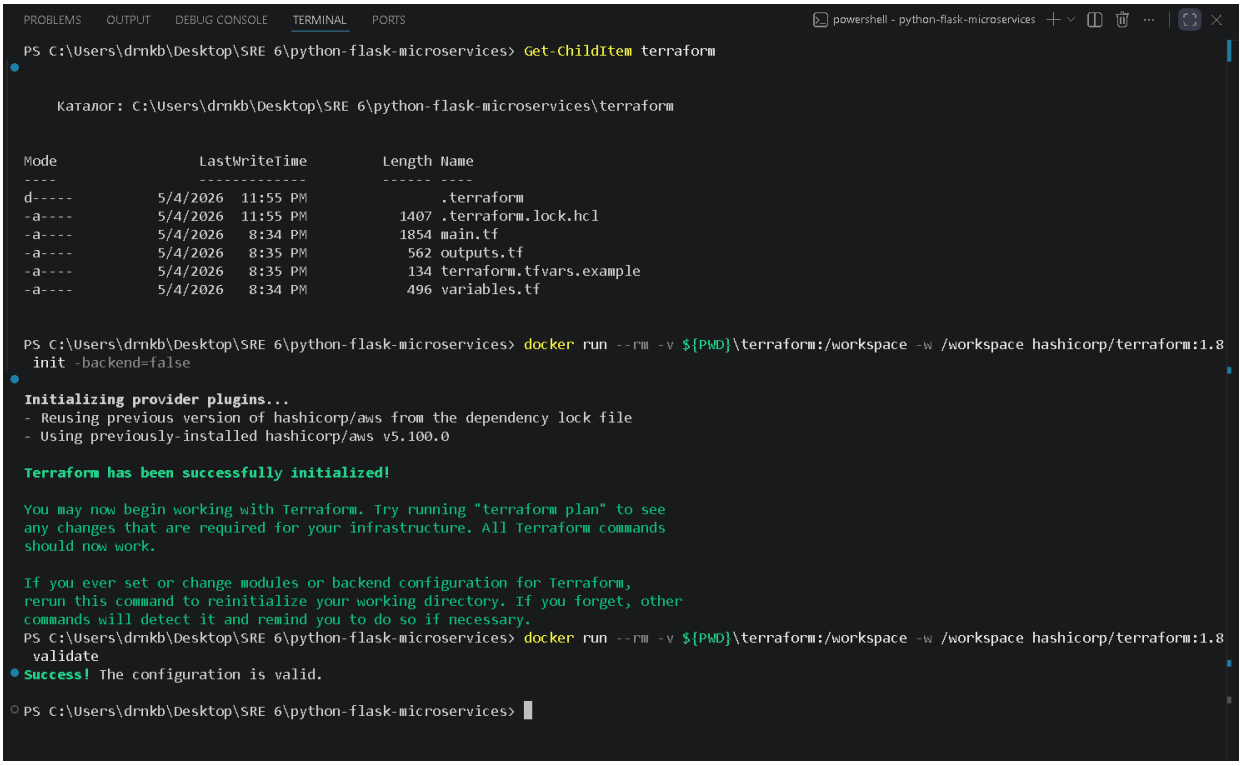

Figure 28. Terraform init and validate succeed through the Terraform Docker image.

Evidence boundary. The report claims Terraform configuration presence and validation. It does not claim terraform apply or live cloud resource provisioning.

14. Scaling Strategy and Capacity Actions

Scaling actions are tied to observed behavior rather than generic statements. The first capacity risk is p95 latency growth, followed by resource saturation and restart instability.

Trigger	Recommended action
p95 latency > 300 ms for 5 minutes	Scale Product and Order services horizontally; add load balancing for service replicas.
Error rate > 1% for 5 minutes	Trigger incident investigation and scale the affected service.
Container CPU > 70% for 5 minutes	Add a service replica or increase CPU allocation.
Product API latency grows under read load	Use PostgreSQL connection pooling, query optimization, and indexes.
PostgreSQL CPU or memory remains high	Tune DB resources, pool connections, and consider read replicas for read-heavy paths.
RestartCount increases by 2 within 10 minutes	Fire recovery alert, inspect logs, and roll back or redeploy misconfigured service.

Horizontal scaling is most appropriate for stateless Flask services. Vertical scaling can temporarily increase CPU/memory for local or VM deployments. Database optimization is required because Product API /api/products is a PostgreSQL-backed request path. Metric-based scaling is proposed for Kubernetes or cloud orchestration but is not claimed as implemented in Docker Compose.

15. Integration with Previous Assignments

Assignment 6 builds on the previous incident and infrastructure work. The Order Service recovery test directly addresses the earlier failure scenario by showing that an application crash can be detected and recovered using Docker restart policy, health checks, and post-recovery validation. Terraform files connect the local runtime to the Assignment 5 infrastructure-as-code requirement.

Previous assignment	Assignment 6 improvement
Assignment 4: Order Service incident	Added health checks, log inspection, validation, alerting, and controlled recovery proof for Order Service.
Assignment 5: Terraform / IaC	Terraform directory is present and validates successfully; scaling strategy includes VM/cloud resource expansion.
Operational reliability	The system now has validation before deployment, monitoring during runtime, and recovery evidence after failure.

16. Limitations and Evidence Boundaries

Boundary	How it is handled
Production benchmark	Load testing is a controlled local simulation, not a production benchmark.
Terraform apply	Terraform init/validate is proven; cloud apply is not claimed.
Auto-scaling	Metric-based auto-scaling is proposed as strategy, not implemented in Docker Compose.
Business UI completeness	Report validates SRE behavior and product-list path. Login/checkout completeness is outside the core Assignment 6 evidence.
Failure endpoint	/crash is controlled failure-injection test instrumentation and should not exist in production.
Boundary test variability	Local Docker Desktop timings are noisy; capacity boundary is interpreted conservatively using SLO breach risk.

Conclusion. The final system satisfies the Assignment 6 intent within the stated local scope: automated deployment, health checks, self-healing recovery, Prometheus/Grafana observability, alert validation, log inspection, PostgreSQL data path validation, capacity analysis, SLO-based scaling recommendations, and Terraform configuration validation.

17. Submission Checklist

The final archive should include the active PostgreSQL-based files and exclude obsolete backup artifacts.

Submission item	Status
frontend, user-service, product-service, order-service source directories	Include
Root docker-compose.yml with PostgreSQL 15, health checks, service_healthy dependencies, and restart policies	Include
monitoring/prometheus/prometheus.yml and alert_rules.yml	Include
nginx/default.conf	Include
README.md and DEPLOYMENT_GUIDE.md	Include if available; README is cleaned of MySQL references
validate_config.ps1, check_logs_clean.ps1, capacity_matrix.py, capacity_boundary_test.py, self_healing_postgres_demo.ps1	Include
terraform/main.tf, variables.tf, outputs.tf, terraform.tfvars.example	Include
Final PDF report and final screenshots folder/zip	Include
_not_for_submission, obsolete MySQL compose files, failed screenshots, old PDF drafts	Exclude

Appendix A. Evidence Inventory

The report uses the screenshot evidence listed below. Exact duplicate or failed screenshots are intentionally excluded.

Evidence group	Figures
Runtime and PostgreSQL path	Fig. 2-7
Validation and logs	Fig. 8-9
Configuration evidence	Fig. 10-12
Prometheus/Grafana/cAdvisor	Fig. 13-18
Load and capacity behavior	Fig. 19-25
Recovery	Fig. 26
Terraform	Fig. 27-28

This inventory supports auditability without turning the report into a raw screenshot dump. Each major screenshot is referenced by the relevant analysis section.